

长上下文场景中 LLM 自动数据标注技术报告

TrailBlazers

2026 年 5 月

摘要

本方案面向“长上下文场景中 LLM 自动数据标注挑战赛”，在只使用 Qwen3-4B、只使用官方数据、通过 FlagScale 推理且不进行微调的约束下，构建了一套覆盖 Task1 到 Task8 的自动标注系统。系统把八个任务拆成规则求解、词性统计、语义分类、开放短答案和 PyTorch fallback 代码生成五类问题，再分别使用长上下文 ICL、候选路由、可执行验证和定向修复来稳定输出。

最终方案的核心不是单个 prompt，而是一套分层推理框架：底层由 FlagScale + vLLM + Ascend910 双卡服务承载 Qwen3-4B；中层按任务构造 30K/16K 长上下文、候选分支和路由器；上层进行上下文审计、Task8 执行验证、JSONL 规范化和 zip 打包。历史最高线上提交达到约 86.79 分。

1 总体方案设计

1.1 总方案概括

本方案采用“统一框架、任务分型、可验证闭环”的设计。统一框架保证所有任务都由同一个 Qwen3-4B 服务和同一套提交管线产生结果；任务分型让不同任务使用不同的上下文组织、提示约束和候选选择方式；可验证闭环把规则题和代码题从一次性文本生成转化为可检查、可修复、可复现的自动标注流程。

系统总流程如下：

1. 读取当前提交目录中的官方输入副本 input/，包含任务定义、examples 和 test samples。
2. 按任务类型构造长上下文 ICL，Task1–Task7 的主答案生成阶段满足约 30K tokens，Task8 初始代码生成满足约 16K tokens。
3. 对规则任务生成并验证 Python solver；对分类和短答案任务生成多个候选并用 Qwen3-4B 路由；对代码任务执行 pass@k、静态检查、smoke test、执行验证和 repair。
4. 写出 openseek-\{1..8\}-v1.jsonl，再由 src/prepare_submission.py 统一规范化并打包为平台 zip。
5. 记录 context audit、Task8 validator summary、运行日志和消融结果，用于复现审计。

1.2 关键术语说明

为避免技术名词悬空，本文先对后文反复出现的术语作简要说明。ICL 指 in-context learning，即不更新模型参数，而是在提示词中放入任务说明和示例，让模型在当前上下文里临时学会任务格式和边界。solver synthesis 指让模型先生成一个可执行求解函数，再由程序运行该函数产生答案；

它适合 Task1/3/4 这类规则确定、可验证的任务。**router** 指候选路由器，即模型不再自由作答，只在多个候选答案中选择一个，从而降低格式错误和过度生成。**calibrated router** 是带校准规则或校准样例的 **router**，用来减弱标签先验、示例顺序和常见 **token** 偏好。

pass@k 指同一测试样本采样 **k** 个候选，再从候选中选择最可靠结果；它适合代码生成，因为单次生成容易出现函数名、签名或 **API** 错误。**validator** 指验证器，本文中特指静态检查、编译、函数名匹配和运行测试组成的程序化检查链。**smoke test** 指最小可运行测试，即不追求覆盖所有边界，只先确认代码能被调用、返回基本合法结果且不会立即崩溃。**repair** 指带错误信息的自动修复轮次，系统把 **validator** 发现的失败原因反馈给模型，让模型重写或修补候选。**MMR** 指 **maximal marginal relevance**，即在选择示例时同时考虑“与当前样本相关”和“与已选示例不要重复”，用于 Task7 控制检索噪声。

1.3 任务拆解

Task1 closest integers

输出：单个整数。难点：规则归纳、整数抽取。策略：solver synthesis/debug loop。

Task2 count nouns/verbs

输出：单个整数。难点：词性边界、复合词、动名词。策略：四路长上下文候选 + **calibrated router**。

Task3 Collatz

输出：整数序列。难点：规则执行、列表字符串格式。策略：solver synthesis/debug loop。

Task4 concat strings

输出：字符串。难点：字符级保真、顺序与标点。策略：solver synthesis/debug loop。

Task5 tweet sadness

输出：Sad/Notsad。难点：作者情绪归因、讽刺、引用。策略：A/B labeler + **guarded router**。

Task6 MNLI same genre

输出：Y/N。难点：**genre anchor** 与语义蕴含边界。策略：**genre-aware** 候选 + **router**。

Task7 Jeopardy answer

输出：小写短答案。难点：**canonical entity**、冠词、短答案。策略：**reasoning bank** + **semantic MMR** + **router**。

Task8 kernel fallback

输出：Python 代码。难点：函数名、**API**、**shape/dtype/device**、可执行性。策略：**pass@k** + **validator** + **repair**。

1.4 框架中的三类算法策略

可执行规则求解

Task1/3/4 默认不是少样本求解，而是在 **many-shot** 长上下文 **examples** 上让模型归纳 **solve(input_text)**，再用 **held-out validation examples** 执行验证和 **debug**，通过后批量预测测试集；同时保留 **longctx_**

manyspot 可选直接标注分支。关键逻辑位于 `src/main.py` 的 `_run_rule_task_solver_synthesis`、`_select_solver_min_context_examples` 和 `src/method.py` 的 `solver synthesis/debug prompt`；可选分支为 `src/run_manyspot.py` 与 `src/run_task4_budget_ensemble.py`。默认 `solver+many-shot` 分支最稳，历史本地一致性复验中三项规则题均达到 500/500。

多候选路由

Task2/5/6/7 由不同 `prompt`、检索配置或 A/B labeler 产生候选，再让 Qwen3-4B 只在候选字母之间选择。精简提交包不再包含 Task2 候选快照，Task2 默认使用 `TASK2_BRANCH=legacy_router`，由 `src/task2_candidate_router.py` 对 `context_rerank`、`noun_biased`、`noun_rules`、`bucket_balanced` 四路重新推理候选做 `calibrated router`；Task5/6/7 对应 `src/task5_candidate_router.py`、`src/task6_candidate_router.py` 与 `src/task7_candidate_router.py`。消融显示候选分支能暴露系统性偏差，正式策略采用保守路由，避免无条件覆盖强分支。

生成-验证-修复闭环

Task8 多温度 `pass@k` 生成代码候选，经过静态检查、编译、`smoke` 和执行验证后选择，失败样本进入 `repair`。核心脚本为 `src/task8_pytorch_passk.py` 与 `scripts/run_task8_pytorch_passk.sh`；`clean` 全量运行最终 `validator` 达到 166/166 `smoke` 与 `exec` 通过，静态违规为 0。

这三类策略相互补充：`solver` 负责可确定规则，`router` 负责不可程序验证的语义边界，`validator/repair` 负责代码类任务的可执行一致性。它们共同把 Qwen3-4B 的生成能力限制在可审计、可回滚、可复现的工程管线内。

1.5 方法设计依据与文献关联

本方案的长上下文设计首先来自 `in-context learning` 的基本事实：大模型可以在不更新参数的情况下，通过任务说明和输入输出示例获得临时任务能力 [1]。本赛题又明确要求长上下文，因此我们没有把示例压缩成少量 `prompt`，而是尽量使用 30K/16K 的官方 `examples` 来提供充分的格式、边界和数据分布信号。Many-shot ICL 的研究表明，示例数量从 `few-shot` 扩展到 `hundreds/thousands shots` 时，在生成和判别任务上通常能进一步强化任务规范 [2]，这与本方案“多示例定义任务、少自由发挥”的思想一致。

但长上下文并不等于简单堆满示例。相关研究指出，模型对长上下文中间位置的信息利用并不稳定，相关信息在开头或结尾时往往更容易被使用 [3]。因此，本方案把任务定义和共享示例前缀放在前部，把当前样本、动态相似示例和输出契约放在靠近生成位置的尾部；Task5/6 的 24K 共享前缀用于 KV cache 复用，动态尾部则用于弥补样本相关性。示例选择方面，我们参考了 `retrieval-based ICL` 的结论：与测试样本更相似的 `examples` 往往比随机 `examples` 更有信息量 [4]，因此 Task2/5/6/7 都不是无脑截断，而是按词性目标、标签边界、`genre anchor` 或语义 MMR 组织上下文。

分类任务还存在标签先验和近因偏置。Zhao 等指出，少样本提示会受标签频率、示例顺序和常见 `token` 偏好影响 [5]。这正是 Task5/6 不能只靠一个长 `prompt` 的原因：Sad/Not sad、Y/N 都有明显分布偏斜，直接生成容易被上下文中多数标签牵引。因此我们采用 `balanced examples`、A/B 标签压缩、保守主分支和候选路由，把开放生成变成受限选择。多候选思想也与 `self-consistency` 的“生成多条候选路径再聚合答案”相近 [6]，但本方案没有把多数票作为唯一准则，而是在 Task8 这类可执行任务上用 `validator` 选择，在 Task2/5/6/7 上用低触发 `router` 纠偏。

工程结构上，本方案更接近“可编译的 LLM pipeline”而不是单 prompt。DSPy 将 LM 调用组织成可组合、可优化、可度量的程序图 [7]；本项目虽然没有引入 DSPy 框架，但采用了相同的工程思想：把任务拆成示例选择、候选生成、路由、静态检查、执行验证、修复和打包审计等模块，每一层都有明确输入输出和失败信号。这样写报告时也能解释为什么某项技术适合某个任务，而不是只描述“用了长上下文”。

2 框架搭建与运行流程

2.1 服务层搭建

推理服务使用 configs/llm_config.yaml，由 scripts/start_service.sh 按 FLAGSCALE_DIR 指向的 FlagScale 框架调用 run.py 拉起 OpenAI-compatible 服务。默认配置见表 1。

表 1: FlagScale/vLLM 服务配置

配置	值	作用
served_model_name	Qwen3-4B	保证所有 LLM 调用均指向指定模型
dtype	bfloat16	默认开启 bf16 加速，降低显存占用
tensor_parallel_size	2	使用双卡 Ascend 910 切分模型与 KV 负载
max_model_len	32768	支撑 Task1–Task7 的 30K ICL 与 Task8 的 16K ICL
enable_chunked_prefill	true	分块处理长 prompt prefill
enable_prefix_caching	true	对共享长前缀复用 KV cache
max_num_seqs	16	支撑 Task8 pass@k 多并发生成

服务启动后，主流程通过 API_BASE=http://127.0.0.1:2026 调用 /v1/chat/completions，算法代码与模型服务解耦。

2.2 推理编排流程

一键入口为：

```
STAMP=official_full_$(date +%Y%m%d_%H%M%S)\
FRESH_RUN=1\
CLIENT_CONCURRENCY=2\
ROUTER_MAX_WORKERS=4\
T7_MAX_WORKERS=4\
T8_MAX_WORKERS=16\
bash run_full_inference.sh
```

run_full_inference.sh 的编排顺序如下：

1. 检查 Qwen3-4B API、NPU 状态、输出目录和上下文预算。
2. 运行 Task1/3/4；默认使用 solver synthesis/debug loop，也可通过 TASK134_BRANCH=longctx_manyshot 切换到迁入的长上下文 many-shot 候选分支。

3. 运行 Task2 四路候选与 calibrated router；精简提交包默认 TASK2_BRANCH=legacy_router，不依赖保存的候选快照。
4. 运行 Task5 baseline/strict/A-B 多分支，随后两阶段 guarded router。
5. 运行 Task6 anchor、genrefirst、hypothesis、conservative 候选并路由。
6. 为 Task7 构建 reasoning bank，进行 semantic MMR validation grid，选择最优配置生成测试候选并路由。
7. 组装 pre-Task8 包，运行 Task8 pass@k、普通 repair、hard repair 和 final validator。
8. 将 8 个 JSONL 复制到 final_raw，调用 src/prepare_submission.py 规范化与打包。

2.3 输出与审计

正式运行产物包括：

```
outputs/upload_ready_${STAMP}/
outputs/upload_ready_${STAMP}.zip
outputs/${STAMP}/context_audit/
outputs/${STAMP}/task8_final_validator/
outputs/logs/${STAMP}.log
```

FRESH_RUN=1 时，脚本要求目标运行目录为空，避免历史中间结果混入当前推理。Task8 repair 只读取当前运行产生的候选与 seed，不读取历史提交包或隐藏标签。verify_submission.py 会检查 8 个 JSONL 的行数、字段、zip 根目录结构，以及 Task8 的 prediction/code 一致性。

3 评分项：上下文构造与示例选择

本方案的上下文策略是“满足最短长上下文要求，同时让有效信息靠近输出位置”。Task1–Task7 的主生成阶段补足约 30K tokens 官方示例上下文，Task8 初始代码生成补足约 16K tokens；在此基础上，按任务使用结构相似、语义相似、标签平衡、MMR 多样性和共享前缀等不同选择策略，而不是简单截取原始示例。

3.1 长上下文组织原则

1. 规则和输出契约前后呼应：任务定义放在 prompt 前部，最终输出约束放在 query 后部。
2. 相关性优先：分类和短答案任务按当前样本检索相似 examples，代码任务按函数名、算子族和 API 风险注入约束。
3. 多样性约束：Task7 使用 semantic MMR，避免检索结果只覆盖表面相似 clue。
4. 标签边界平衡：Task5/6 避免长上下文中某一标签占比过高导致先验偏置。
5. 审计可追踪：context audit 记录每个任务或样本的目标 tokens、实际 tokens、示例数量和是否通过最短上下文要求。

3.2 共享前缀与动态尾部

Task5/Task6 的候选分支多、prompt 长，重复 prefill 成本高。正式配置对 Task5、Task5 A/B、Task6 默认启用 24K tokens 共享 ICL 前缀：

```
CACHE_PREFIX_CONTEXT_TOKENS_TASK5=24000
CACHE_PREFIX_CONTEXT_TOKENS_TASK5_AB=24000
CACHE_PREFIX_CONTEXT_TOKENS_TASK6=24000
```

前 24K tokens 是固定官方示例前缀，便于 vLLM prefix caching 复用 KV cache；尾部继续追加与当前样本相关的动态示例和 query，保证样本相关信息靠近答案生成位置。这样把性能优化和算法精度分开处理，避免为了速度完全牺牲动态检索。

4 评分项：提示策略与输出约束

本方案的提示策略是“共享骨架 + 任务契约 + 输出空间压缩”。所有任务共享任务定义、示例区、当前样本和输出契约的骨架，但每类任务使用不同的输出约束：规则题要求生成可执行 solver，分类题要求单标签，Task7 要求小写 canonical answer，Task8 要求纯 PyTorch fallback 代码且禁止 Triton/CUDA。

4.1 分任务提示契约

Task1/3/4

输出 solve(input_text:str)->str 的纯 Python 函数；不得读文件、调用外部服务或使用随机数。

Task2

只输出整数；默认重新生成 context_rerank、noun_biased、noun_rules、bucket_balanced 四路候选，并用 calibrated router 在整数候选之间选择。

Task5

将 Sad/Notsad 压缩为 A/B 二选一；明确区分作者本人情绪和引用、玩笑、第三人称事件。

Task6

先固定 genre/source anchor，再判断句子关系；避免只看词面重合。

Task7

只输出一个小写短答案；针对冠词保留、最短无歧义答案和标题类答案使用不同 answer rules。

Task8

输出指定函数名的 PyTorch fallback；禁止 Triton、CUDA、placeholder、危险导入和无关解释。

4.2 路由提示

Task2/5/6/7 的 router prompt 不让模型重新自由作答，而是要求读取原输入、候选答案和任务规则后只输出候选字母。Task2 在精简提交包中使用四路重新推理候选：context_rerank 强调上下文相似示例，noun_biased 强化名词边界，noun_rules 注入显式计数约定，bucket_balanced 平衡答案数值

分桶；**calibrated router** 只在这些候选整数之间选择。这样把开放生成问题转化为可审计的受限选择，降低格式错误和过度覆盖强分支的风险。

5 评分项：多轮交互与一致性

本方案的多轮交互不是人工聊天，而是由程序自动触发的闭环：先让模型生成候选，再用任务级检查器发现错误，最后把失败样本、失败原因或候选差异反馈给同一个 Qwen3-4B。这种交互让模型专注修复可定位问题，同时把不可控的重复采样限制在候选选择和验证门控内。

5.1 Solver debug loop

Task1/3/4 的第一轮输出 solver，但这一步同样使用 many-shot 长上下文。run_full_inference.sh 对 Task1/3/4 调用 src/main.py 时带有 --official-min-context，_run_rule_task_solver_synthesis 会通过 _select_solver_min_context_examples 在 token 预算内连续选取官方 examples，直到接近 min_context_tokens 目标；这些样例随后由 build_task_solver_synthesis_prompt 写入 [ReferenceExamples]，作为模型归纳规则和生成 solve(input_text) 的主要证据。

因此默认路径可以概括为“many-shot 归纳规则 + 可执行 solver 验证”，而不是“少样本写程序”。若 solver 在 validation examples 上失败，系统把失败输入、期望输出、当前输出和错误原因写入 debug prompt；debug prompt 仍保留同一批 many-shot reference examples，并要求模型修复完整函数。只有 solver 在验证样本上全通过，才批量预测测试集。

工程上还保留了 longctx_manyslot 可选分支。设置 TASK134_BRANCH=longctx_manyslot 时，run_full_inference.sh 会调用 src/run_manyslot.py 为 Task1/3/4 生成 value-cover anchored reasoning 长上下文候选；Task4 进一步生成 18K、12K、28K、raw transcribe 和 line transcribe 等多预算候选，并由 src/run_task4_budget_ensemble.py 在已有候选之间投票、路由和规则改选。该分支用于复现实验和对照，默认正式分支仍是 solver_debug_loop。

新增代码没有改变正式默认输出路径。TASK134_BRANCH 的默认值仍为 solver_debug_loop；只有显式设置 TASK134_BRANCH=longctx_manyslot 才会进入 many-shot 分支。该分支的 manifest 会记录 no_solver_generation、no_code_execution_for_task_solution 和 no_programmatic_task_answer_computation，说明它不是规则求解器，也不执行任务答案计算，而是纯模型长上下文候选生成。

5.2 候选路由

Task2/5/6/7 的不同 prompt 分支相当于多视角推理。Task2 当前精简提交包不读取历史候选快照，而是现场生成四路候选并由同一个 Qwen3-4B 进行 calibrated router；Task5/6/7 同样由 Qwen3-4B 比较候选，不引入外部模型或外部标签。该设计比 majority vote 更适合本赛题，因为候选数量少但差异具有语义含义，系统可以结合输入结构判断分歧来自哪类错误。

5.3 Task8 pass@k 与 repair

Task8 初始阶段每个样本生成多个候选，再经过静态检查、函数名匹配、编译、smoke test 和执行验证。未通过的样本进入 repair，repair prompt 包含失败原因、期望函数名和 API hint。脚本设置 ACCEPT_ONLY_SMOKE_OK_IMPROVEMENTS=1，避免新候选破坏已有可执行结果。

6 分任务算法设计与样例分析

本节基于新增的官方输入目录和最终输出 JSONL 进行逐项分析。八项任务的规模和性质差异很大：Task1/2/3/4/5/6/7 的测试集均为 500 条，但 examples 数量从 1899 到 5500 不等；Task8 只有 184 条 examples 和 166 条测试样本，却要求输出可执行代码。因此，本方案没有把八项任务统一成一个“长 prompt 分类器”，而是按任务可验证性、输出空间和错误成本分别设计。

6.1 Task1: closest integers 的确定性数值归纳

Task1 给定整数列表，要求输出任意两个整数的最小绝对差。该任务表面上是简单算术，但对 LLM 来说存在两个常见误区：第一，模型容易把“相邻输入元素差”误认为答案，而不是对排序后的所有相邻值求最小差；第二，负数和重复/近邻数字会放大心算错误。由于输出是单个整数且规则完全确定，最适合使用 many-shot solver synthesis，而不是让模型逐条生成答案。

技术上，系统先把任务定义和 many-shot 官方 examples 交给 Qwen3-4B，要求生成 solve(input_text)。随后程序在 held-out validation examples 上执行该 solver，若出现失败就把失败输入、期望输出和实际输出反馈给模型修复。通过验证后，测试阶段只运行 Python 逻辑：解析整数列表、排序、计算相邻差、返回最小值。这种做法把 LLM 的作用限定为“基于 many-shot 归纳规则并写程序”，把 500 条测试样本的标注变成可复现计算。

具体判断流程分三步。第一步用 ast.literal_eval 或等价解析方式把输入安全地转成整数列表，避免用正则误吞负号。第二步对列表排序，只比较排序后相邻两个数，因为一维数轴上最小绝对差必然出现在相邻元素之间。第三步把所有相邻差取最小值并转成字符串输出。validation examples 只要发现一次“按原输入顺序比较”或“漏掉负号”的错误，debug loop 就会把该失败样本放回 prompt，让模型修正 solver。

样例能够说明为什么排序是关键：

输入: [71, -93, 63, -41, -18, 18]

输出: 8

原始顺序里 71 和 63 并不相邻，但排序后相邻，差值为 8；同一批数据中另一个样本 [43, 77, -55, 51, 66, 20] 的输出也是 8，对应 43 和 51。这个案例直接支撑了“先归纳 solver 再批量执行”的设计：规则题真正需要稳定执行，而不是语言生成。

可选的 longctx_manyslot 分支则直接利用长上下文示例生成候选。src/run_manyslot.py 支持 structure_cover 和 value_cover 两种上下文组织方式：前者按输入结构、输出桶和特征桶做 round-robin 覆盖，后者在共享示例前缀之外，把与当前 query 在数值、长度、字符库存或 Collatz 奇偶结构上更接近的 examples 放到靠近输出的位置。正式脚本使用 value_cover，并把 Task1/3/4 的主候选设为 28K token 级别的 anchored_reasoning。

表 2: Task1/3/4 longctx many-shot 可选分支

子流程	任务	关键参数	作用
full_value_cover_anchored_reasoning	Task1/3/4	28K / 30K	长上下文 many-shot 主候选, Task1/3 直接进入 combined 输出
full_t4_18k_value_cover_anchored_reasoning	Task4	18K / 26K	Task4 中等预算候选, 降低过长上下文复制干扰
full_t4_12k_value_cover_anchored_reasoning	Task4	12K / 20K	Task4 短预算候选, 用于纠正边界和字符漂移
full_t4_raw_transcribe	Task4	raw / 512	把输入列表当 raw chunks 转写, 提供字符保真候选
full_t4_line_transcribe	Task4	line / 18K	把字符串列表改写为 numbered chunks, 减少 list 语法干扰
src/run_task4_budget_ensemble.py	Task4	六路候选	只在已有候选中投票、路由或规则改选, 形成 longctx 分支 Task4 输出

Task4 的 ensemble 分两层。第一层 dselect 在 18K、12K、28K 候选不一致时调用 select-only router, 只允许模型返回候选选项字母, 不允许自由改写标签。第二层把 18K、12K、28K、raw、line、dselect 六路候选按权重投票, 再依次应用四类只选已有候选的保护规则: 无空格输入时优先无空格候选、输出长度必须等于输入片段长度之和、字符 multiset 必须一致、相邻字符交换候选必须至少有两路来源支持。最终 Task4 ensemble 不拼接输入、不生成新字符串, 只在模型已经给出的候选之间选择, 因此仍满足“不引入外部模型、不用人工标签选择”的约束。

6.2 Task2: count nouns/verbs 的词性边界识别

Task2 要在 caption 风格句子中统计名词或动词。输入数据中训练 examples 为 5440 条, 目标分布较平衡: verbs 2755 条、nouns 2685 条; 测试集中 nouns 252 条、verbs 248 条。难点不在输出格式, 而在词性边界: 颜色词、数量词、复合名词、动名词、助动词、系动词和分词都可能造成偏差。例如 traintracks 中 train 在语义上修饰 tracks, 但在 caption 计数约定里常被视为名词成分; iscomingdown 则需要把动作谓词识别为动词短语。

因此本方案没有依赖单一 prompt, 而是为 Task2 现场生成四路长上下文候选, 再把正式输出收敛到可审计的候选路由。精简提交包的默认分支为 TASK2_BRANCH=legacy_router, 由 src/task2_candidate_router.py 读取 context_rerank、noun_biased、noun_rules、bucket_balanced 四路候选。router prompt 会展示原句、目标词性、候选整数和若干 caption-counting calibration examples, 并要求模型只输出候选字母, 不允许自由生成新整数。这样保留 Task2 多视角推理的优势, 同时避免在提交包中携带候选快照或历史输出。

示例选择不是随机抽样。系统先解析当前输入, 得到待统计句子和目标词性; 再抽取句长、标点、引号、连字符、数字、ing/ed 后缀、大小写、be/have/do 等特征。候选 example 的评分由三部分组成: 目标词性相同加权最高, 词面重合和句长接近次之, 形态特征相近作为补充。名词题额外提高同目标示例权重, 因为名词边界比动词更容易被颜色词、复合名词和专名影响。bucket-balanced 分支还按答案数值分桶, 例如名词题优先覆盖 2、3、4、5、6 等常见答案, 避免上下文里全是同一

个计数值。

router 的判断也有明确规则。它先识别请求统计 **nouns** 还是 **verbs**，然后按 **caption** 计数约定重新数一遍：名词只数人、物体、地点、物质、身体部位等名词性词，排除冠词、限定词、普通颜色词、代词和介词；动词统计谓词、系动词、助动词和动作性分词，排除只是长得像动词的名词。候选 **A** 被设为当前最强分支，**router** 只有在重数结果清楚支持 **B/C/D** 时才切换，这样可以减少路由器把强主干改坏。

两个测试样例展示了这种规则化提示的必要性：

输入: Sentence: 'A yellow train is coming down some train tracks'. Count the number of nouns in this sentence.

输出: 3

输入: Sentence: 'very types many ripe fruits in a basket'. Count the number of nouns in this sentence.

输出: 3

第一句的有效名词来自 **train/train/tracks**，而 **yellow** 和 **some** 不能计入；第二句的名词是 **types/fruits/basket**，**very/many/ripe** 不是目标词性。这类样例说明 **Task2** 需要把“语言理解”转化为显式计数约定，**noun_rules** 在 **held-out** 消融中成为最强单分支也符合这个任务特点。

6.3 Task3: Collatz 单步变换的列表级保真

Task3 要对列表中每个整数执行一次 **Collatz** 变换：偶数除以二，奇数乘三加一。与 **Task1** 类似，它是完全确定的规则任务；但 **Task3** 的输出是列表，因此还多了顺序、元素数量和 **JSON-like** 字符串格式要求。**LLM** 逐条生成时容易出现“继续迭代到 1”的误读，或者在列表格式上多写解释文本。**solver synthesis** 能把这些风险一次性消除。

最终 **solver** 的核心逻辑很短：解析输入列表，按原顺序遍历每个元素，偶数输出 $n/2$ ，奇数输出 $3*n+1$ ，最后用标准列表字符串返回。验证阶段重点检查两点：输出长度必须等于输入长度；每个元素只做一步变换，不递归执行完整 **Collatz** 序列。

该任务的失败处理也很直接。系统把 **examples** 分成 **solver prompt** 中的归纳样例和本地验证样例，验证时逐条比较字符串规范化后的列表。若模型写出“循环直到 1”的 **solver**，验证样例会立即出现输出长度相同但数值不匹配；若模型输出元组、空格异常或额外解释，**postprocess** 会先抽取可解析列表，仍不合法再触发 **debug**。由于最终批量预测完全由通过验证的函数完成，测试集不会再受采样温度或 **prompt** 顺序影响。

测试样例如下：

输入: [91, 63, 148, 8, 6]

输出: [274, 190, 74, 4, 3]

91 和 63 是奇数，分别得到 274 和 190；148、8、6 是偶数，分别得到 74、4、3。这个例子覆盖了奇偶混合、顺序保持和列表格式，说明 **Task3** 的核心不是推理深度，而是稳定执行确定性映射。

6.4 Task4: 字符串拼接的字符级复制

Task4 给定字符串列表，要求从左到右拼接所有元素。该任务看起来比自然语言任务简单，但对 **LLM** 是典型的字符级保真问题：大小写、标点、数字、空字符串、单字符片段和多字符片段都

必须原样保留。错误不一定来自规则不懂，而来自生成模型的“自动规范化”，例如补空格、改大小写、漏掉短片段或把列表语法复制到答案里。

因此 Task4 使用两层策略。第一层仍是 **solver synthesis**：让模型写出解析 Python list 并执行 join 拼接的程序，通过 **examples** 验证后批量执行。第二层保留严格 **prompt** 与 **review/retry** 工具作为兜底，提示中明确要求“只删除列表语法、引号、逗号和元素间分隔空格，不改变元素内部字符”。在实现上，Task4 的示例选择还会考虑列表长度、元素长度、大小写和标点等结构相似性，因为这些因素比语义相似更能预测字符级错误。

结构相似度的计算包含可解释的字符统计：元素个数、拼接后总长度、单字符元素数量、多字符元素数量、大写字母、小写字母、数字和标点数量。当前样本如果有大量单字符片段，系统就优先放入同样“碎片化”的 **examples**；如果有大小写混合或标点，系统优先放入类似 **examples**。判断输出时不看语义，只检查拼接长度和字符统计是否与输入元素总和一致；**review prompt** 也是围绕“漏字符、加空格、改大小写、复制列表语法”这几类错误设计。

测试样例：

输入：['f', 'k', 'buttoned', 'e', 'are', 'f', 'I', 'as', 'W']

输出：fkbuttonedeareflasW

这里 I 和 W 必须保持大写，buttoned/are 等词之间不能补空格。该样例说明 Task4 的技术重点是“复制精度”，不是语言理解；因此可执行 **solver** 比更长的自然语言解释更合适。

6.5 Task5: tweet sadness 的作者情绪归因

Task5 是二分类，但不是普通情感分析。任务定义要求判断 **tweet** 作者是否悲伤，可以参考 **hashtag** 和 **emoji**，但不能只看表面情绪词。输入 **examples** 中 Sad 为 1121 条、Notsad 为 778 条，存在一定标签偏斜；测试输出中最终预测为 Sad 313 条、Notsad 187 条。难点集中在三类边界：引用或歌词里出现负面词、第三人称事件很悲伤但作者未表达自身情绪、以及讽刺/吐槽/愤怒是否应算作 **sadness**。

本方案将标签压缩为 A=Notsad、B=Sad，分别生成 **conservative**、**balanced**、**strict** 和 A/B **labeler** 候选，再用 **guarded router** 做低触发纠偏。这样设计是因为 Task5 的错误成本不对称：如果只追求 Sad 召回，引用、新闻和玩笑会被大量误判；如果过度保守，真实第一人称痛苦会漏掉。A/B 形式可以减少标签 **token** 偏好，**balanced examples** 可以降低长上下文多数标签偏置，**guarded router** 则专门检查“作者本人是否正在表达负面状态”。

示例选择时，系统先按词面相似度召回，再按情绪线索分组加权。线索组包括 **quote/joke/lyrics**，**lost/miss**，**hurt/pain/sleep**，**pissed/fuming/frustrated**，**sad/depress/lonely**，以及 **politics/news** 等。保守分支的排序循环采用 Notsad, Notsad, Sad，让引用、玩笑和评论类负例在上下文中占足比例；**balanced** 分支采用 Notsad, Sad 交替，防止模型只记住负例。这样做的目标不是改变标签先验，而是让模型在关键边界上看到对比样例。

guarded router 的决策过程可以概括为两问。第一问：**tweet** 作者本人是否表达了当前负面状态，例如丢失重要物品、身体疼痛、睡不着、沮丧、愤怒或孤独。如果是，即使语气口语化或带 lol，也倾向 Sad。第二问：负面词是否只出现在引用、歌词、玩笑、政治/体育评论、第三人称事件或对他人的辱骂中。如果是，保持 Notsad。候选 A 仍是强主干；**router** 只有在另一候选能清楚解释作者本人 **distress** 时才切换。

两个输出样例展示了判断边界：

输入: Always do sober what you said you'd do drunk. ... Ernest Hemingway #quote

输出: Not sad

输入: @SizweM01 and it's kinda depressing hey!!!

输出: Sad

第一条虽然包含 sober/drunk/keepyourmouthshut 等负面或告诫语气，但它是引用格言，作者没有表达自身悲伤，所以输出 Notsad。第二条直接用 depressing 描述当前交流对象或处境，且是作者自己的即时评价，因此更接近 Sad。这说明 Task5 不能只靠情绪词表，而必须把“谁在感受情绪”作为首要判断轴。

6.6 Task6: MNL same genre 的 genre-anchor 判断

Task6 输入两句话和一个 genre，输出 Y/N。训练 examples 中 N 为 3703 条、Y 为 1797 条；五个主要 genre 分布相对均衡，包括 telephone、government、travel、fiction 和 slate。该任务容易被误解为普通语义相似度或自然语言推理，但数据形式更接近“同 genre 下的 same-source/hypothesis 判断”：sentence 2 可以是 sentence 1 的简化、推论、矛盾或续写，但必须围绕同一主要对象/事件，并且不能明显脱离给定 genre。

因此 Task6 的主干是 anchor-first。系统先生成 anchor、genrefirst、hypothesis、conservative 四类候选，再用 router 在候选间选择。anchor prompt 要求先找 sentence 1 的主实体、场景或事件，再判断 sentence 2 是否仍围绕该 anchor；genrefirst prompt 则先排除 genre 明显不匹配的句子。这样的顺序很重要：只看 genre 会把两个同风格但无关的句子误判为 Y；只看词面相似又会忽略短 hypothesis 的合理性。

上下文构造也围绕 genre 和标签平衡展开。系统先解析出 sentence 1、sentence 2 和 genre；示例评分时，正文词面重合和长度接近提供基础分，同 genre 额外加权，不同 genre 降权。随后在同 genre 池和其他 genre 池内分别按 N,N,Y 的循环排列 examples。这是因为训练集中 N 明显多于 Y，而真实判断又需要大量负例展示“同 genre 但不同 anchor”的情况；少量 Y 样例则用来说明短 hypothesis、矛盾或简化句仍可能成立。

router 判断时按三层门控执行。第一层看 genre：telephone 要有口语对话痕迹，government 要偏正式制度/政策/管理文本，fiction 要有叙事或人物动作，travel 要像地点介绍，slate 要像评论/观点文。第二层看 anchor：sentence 2 是否还围绕 sentence 1 的主体、地点、事件或情境。第三层看 relation：简化、蕴含、反驳、细节变化都可以是 Y，但换到新的主体或只共享泛泛主题词就是 N。这个判断顺序能解释为什么普通语义相似度不足以解决 Task6。

测试样例说明了该边界：

输入: Sentence 1: Once Caterpillar has established this plan, it tracks demonstrated reliability against it ...

Sentence 2: People in a place. Genre: government.

输出: N

输入: Sentence 1: The man watched the stream fall.

Sentence 2: The stream was quiet and peaceful. Genre: fiction.

输出: Y

第一组的 sentence 1 有企业管理/可靠性跟踪语境，sentence 2 是过短的泛化片段，既没有共享 anchor，也缺乏 government 文体证据，因此为 N。第二组都围绕同一条 stream 和同一叙事场景，sentence 2 可以作为 fiction 风格的续写或描述，因此为 Y。该案例解释了为什么 held-out 消融中 anchor 分支能达到 104/120，并成为正式主干。

6.7 Task7: Jeopardy 短答案的检索、规范化与候选选择

Task7 是开放短答案生成。官方 examples 有 5499 条，但 category 极其分散，训练集中最高频 category 也只有十几条；最终测试输出的平均答案长度约 1.68 个词，最长 8 个词。这意味着任务不是简单分类，也不是只靠 few-shot 格式即可解决。模型必须综合 category、clue、常识和 Jeopardy 答案格式，输出小写 canonical answer；同时还要避免过长解释、错留冠词、答案同义词不规范等问题。

本方案坚持只使用官方 examples，不接外部知识库。技术路线是先构建 reasoning bank：对 examples 抽取 category 类型、clue 触发词、答案形式和可能的推理模式；随后为测试样本生成语义标签，用 semantic MMR 检索既相关又多样的 examples。MMR 的作用是避免上下文被同一表面主题占满，例如 category 相似但 clue 机制不同的样本会误导答案格式。validation grid 进一步搜索 answer rules、MMR lambda、candidate pool、retrieval_k 和 reasoning 截断长度，最后用 candidate router 选择。

semantic tag 的字段是可解释的：question type 表示 clue 在问人物、地点、作品还是词义；answer type 表示答案应是人名、地名、普通名词、缩写或标题；knowledge domain 表示历史、文学、科学、地理等领域；reasoning style 表示是否需要双关、年代线索、类别约束或实体识别；constraints 和 risk flags 记录冠词、大小写、简称、同义答案等风险。检索时先计算这些字段的加权重合，再加入少量词面重合；MMR 选择公式是“相关性乘以 lambda，减去与已选案例的冗余度”，因此能保留一个最相关案例，再补入推理风格不同的案例。

候选生成后还要做答案规范化判断。answer rules 有两类：shortest 分支倾向去掉 a/an/the 并输出最短无歧义答案；preserve-articles 分支在标题、固定短语和命名作品中保留冠词。validation grid 发现 preserve-articles 在当前切分更优，因此正式配置采用它作为主规则。router 选择时重点比较候选是否过长、是否把 clue 解释写进答案、是否错误删除标题冠词、是否把相邻实体当成答案；它不是检索外部知识，而是基于官方 reasoning bank 和候选差异做最终规范化。

一个典型样例：

输入: Category: BRITISH AUTHORS

Clue: After a fatwa was issued against him in February 1989, he went into hiding under police protection

输出: salman rushdie

这里 category 把搜索空间限制为英国作家，clue 中的 fatwa/February1989 是强触发线索，输出需要是 lower-case 人名而不是完整句子。该样例说明 Task7 的策略不能只检索表面相似文本，还要保留“线索触发词到 canonical entity”的推理结构，并通过 answer rules 控制最终形式。

6.8 Task8: kernel generation 的 PyTorch fallback 验证闭环

Task8 的输入要求生成 Triton kernel 和 wrapper，但比赛约束与本地验证环境决定了更稳妥的目标是 CPU-safe PyTorch fallback：只要公开 wrapper 的函数名、签名、参数语义、shape/dtype/out

行为正确，就能避免 Triton/CUDA 依赖带来的不可执行风险。该任务只有 184 条 examples、166 条测试样本，且输出是代码；任何一个函数名错误、非法 import、placeholder、签名不匹配或运行时异常，都会直接造成该样本失败。因此 Task8 必须使用 pass@k、静态门控、执行验证和 repair，而不是一次生成。

具体流程如下。首先从 WrapperEntryInformation 中抽取期望函数名，并按函数名注入 API hint，例如 top-level torch、torch.linalg、torch.special、conv/pooling/normalization/loss/dropout 等。生成阶段使用多温度 pass@k 产生多个候选；选择阶段依次检查禁止 Triton/CUDA、禁止不存在的 F.* API、禁止错误 kwargs、函数名匹配、Python 编译、命名空间执行和 CPU smoke test。对于 addmm/rand/repeat 等高风险 API，还加入额外 smoke。失败样本进入 repair prompt，repair 只接受从 non-smoke-ok 改进到 smoke-ok 的候选，避免破坏已有可执行代码。

候选排序不是只看模型置信度，而是按程序化信号分层。第一层排除静态违规，例如导入 Triton、使用 CUDA-only API、出现 pass/placeholder、调用不存在的 F.sqrt 或传入 PyTorch 不支持的关键字参数。第二层检查候选是否定义了期望 wrapper；如果目标来自 linalg 命名空间，函数名要取最后一段真实 API 名，不能臆造前缀。第三层执行编译和命名空间加载，捕获语法错误、NameError 和缺失导入。第四层根据函数签名合成小张量、标量、dtype、dim、out 等参数做 smoke test，确认返回类型和基本 shape 合理。最终选择优先级是 smoke ok、函数名匹配、静态问题少、错误信息短。

输出样例可以看出为什么函数名和签名是第一优先级：

输入功能: Divides each element of input by other ...

期望 wrapper: div

输出开头: def div(input, other, *, rounding_mode=None, out=None):

如果模型写成 torch_div、漏掉 rounding_mode，或忽略 out，即使主体计算近似正确也会在调用处失败。对于 fused bmm + RMSNorm + GELU + dropout + sub 这类复合算子，系统更强调“可读 PyTorch 顺序实现”而不是伪 Triton skeleton：先用 torch.bmm 获得中间张量，再按 wrapper 语义执行 norm、activation、dropout 和 subtraction。最终 clean 全量运行中 Task8 validator 达到 166/166 smoke 与 exec 通过，说明这种生成-验证-修复闭环比单次代码生成更适合代码型标注任务。

7 实验结果、消融与分析

7.1 全量运行质量信号

当前 clean 全量运行的预测包为：

build/submission.zip

表 3: clean 全量运行关键质量信号

项目	结果
Task1–Task7 答案生成上下文	均满足 30K ICL context 目标
Task8 初始生成上下文	166/166 样本满足 16K ICL context 目标
Task8 最终 smoke test	166/166 通过
Task8 最终执行验证	166/166 通过
Task8 函数名匹配	166/166 通过
Task8 静态违规	0 条
最终 zip 格式	8 个根目录 JSONL，无嵌套目录

这些指标不能替代隐藏测试评分，但说明全流程满足长上下文、可执行性和提交格式要求。

7.2 八任务消融设置与总体结论

补充消融由本地脚本组织复现实验设置，表中整理本地跑测结果和官方 examples held-out，不使用隐藏测试标签。Task2/5/6 使用官方 examples 的确定性 held-out; Task1/3/4 使用仓库参考输出一致率做本地验算；Task7 当前批次生成 7 组全量预测并引用历史官方 validation grid；Task8 当前批次做 sample20 k=1/4/8，并引用历史全量 pass@k/repair validator。精简提交包已删除消融和报告渲染辅助脚本，正式推理链路保留 Task2 四路候选 router、Task5/6/7 候选路由和 Task8 pass@k repair。

实验环境保持统一：所有候选生成、路由和修复均使用 Qwen3-4B，推理服务为 FlagScale+vLLM，硬件为双卡华为昇腾 910，总显存约 128GB。实验过程不使用外部模型、不微调、不读取隐藏测试标签，也不把消融输出反向写入正式测试预测。不同任务使用的评估信号按任务类型区分：Task1/3/4 报告 500 条本地一致性验算 accuracy；Task2/5/6 报告官方 examples held-out 的 Accuracy@120，并补充混淆矩阵、FPR/FNR、Y/N 分项或 MAE/off-by-one；Task7 报告官方 validation grid 的 Accuracy@120；Task8 报告 smoke、exec、static gate 和函数名匹配等可执行验证指标。

消融设计遵循四个原则。第一，完整性：每类任务至少保留一个完整主流程和一个关键组件移除变体。第二，可解释性：同一组对比尽量只改变一个核心机制，例如是否启用 solver、是否提高 Sad 召回、是否增大检索 k。第三，工程可行性：debug loop、validator 和 repair 这类保险组件即使在某个切分上没有带来额外准确率，也保留其防回归价值。第四，统计可解释性：分类任务不只报告 accuracy，还补充 TP/TN/FP/FN、FPR/FNR 或 Y/N 分标签结果，避免单一准确率掩盖错误成本。

表 4: 消融实验设计原则与变量

消融方法	适用任务	核心目的	消融变量
组件逐一移除	Task1/3/4/6	拆除核心组件，量化单组件贡献	solver、debug loop、anchor 信号
上下文窗口消融	Task7	验证检索感受野对 canonical answer 的影响	retrieval k、MMR lambda、pool、char limit
标注粒度消融	Task3/4	分离规则理解、列表/字符格式与输出规范误差	是否提供格式提示、是否程序化执行
上下文特征源消融	Task2/4	比较上下文特征、分桶和候选源贡献	context、noun rules、bucket、structured examples
标注传播策略消融	Task5/8	量化验证闭环和标签边界策略价值	pass@k、repair、router 守卫强度

表 5: 八任务消融覆盖与关键结论

任务	消融覆盖	关键结论
Task1	full / no debug / no solver	solver 是主要收益来源，去掉 solver 降到 86/500
Task2	context / noun / rules / bucket / router	候选差距集中在 1-2 题，支持保守 calibrated router
Task3	ICL list / solver / solver+debug	solver 消除 Collatz 执行与列表格式风险
Task4	baseline / strict / structured / solver	字符任务必须可执行化，prompt-only 不稳定
Task5	baseline / strict / conservative / recall / router	保守 A/B 边界最佳，单纯召回会增加误报
Task6	anchor / genrefirst / hypothesis / conservative / router	anchor-first 最稳，router 不破坏主干
Task7	k/lambda/pool/char grid	k=2 的少量高质上下文优于更多检索样例
Task8	sample20 k=1/4/8、full/repair/final	收益来自 pass@k + validator + repair 闭环

表 6: 八任务消融总体结果汇总

任务	最优变体	最优结果	关键对照	主要差异
Task1	full_solver_debug_loop	500/500	no_solver_icl: 86/500	+414 条
Task2	noun_rules / router	68/120	noun_biased: 65/120	+3 条
Task3	full_solver_debug_loop	500/500	no _ solver _ icl _ list: 81/500	+419 条
Task4	full_solver_debug_loop	500/500	structured _ examples: 236/500	+264 条
Task5	ab_conservative_k8	87/120	strict_t0: 73/120	+14 条
Task6	anchor / router	104/120	conservative: 95/120	+9 条
Task7	k=2,lambda=0.70,pool=24	47/120	k=3,lambda=0.70,pool=24: 36/120	+11 条
Task8	final_validator	166/166	full _ passk _ initial: 142/166 smoke	+24 条 smoke

从总体结果看，规则任务和代码任务的收益最集中：Task1/3/4 只要转为可执行 solver，500 条本地验算均能达到完全一致；Task8 则从初始 142/166 smoke 通过提升到最终 166/166。分类和开放问答任务的增益更依赖边界控制：Task2 各分支只差 1–3 条，说明瓶颈在词性边界；Task5 的主要收益来自降低 Not sad 误报；Task6 的 anchor-first 明显优于保守语义相似；Task7 的 k=2 配置优于更大检索窗口，说明开放短答案存在上下文噪声拐点。

7.3 Task1/3/4 solver 消融

表 7: Task1/3/4 solver 消融

Task	variant	agreement	correct/total
Task1	full_solver_debug_loop	1.0000	500/500
Task1	no_debug_loop	1.0000	500/500
Task1	no_solver_icl	0.1720	86/500
Task3	full_solver_debug_loop	1.0000	500/500
Task3	no_debug_loop	1.0000	500/500
Task3	no_solver_icl_list	0.1620	81/500
Task4	full_solver_debug_loop	1.0000	500/500
Task4	random_or_prompt_baseline	0.5560	278/500
Task4	strict_prompt_only	0.5040	252/500
Task4	structured_examples	0.4720	236/500

这组消融说明，Task1/3/4 的核心不是“是否使用 many-shot”，而是 many-shot 之后如何落地：默认 full_solver_debug_loop 本身已经在长上下文 reference examples 上归纳规则，关键收益来自把归纳结果转成可执行 solver，并用 validation examples 做本地闭环验证。Task1 和 Task3 的 solver 一次生成已经达到 500/500；Task4 只有 solver/debug 能稳定保持大小写、标点和短片段顺序，非 solver prompt 组只在 47.2%–55.6% 区间。

错误模式也能解释 solver 的决定性收益。Task1 的无 solver ICL 往往把数组趋势解释成自然语言结论，或者按输入原顺序比较相邻元素，而不是排序后找最小绝对差；Task3 的无 solver ICL 常见问题包括奇数操作误用、是否在 1 处停止不一致，以及输出步数而非逐元素单步变换列表；Task4 的 prompt-only 分支容易做语义合并、去重、补空格、首字母提取或大小写归一化。这些错误不是格式提示能完全修复的输出问题，而是“文本生成”与“确定性执行”之间的机制错配。

Debug loop 在 Task1/3 的当前切分上没有带来额外 accuracy，完整分支和 no_debug_loop 都为 500/500。但它仍被保留为工程保险：当 solver 生成阶段出现边界条件、输出格式或解析失败时，debug loop 可以把失败样本、期望输出和实际输出重新反馈给同一个模型，避免一次采样失败直接污染批量预测。

因此最新代码保留 longctx_manyshot，但不把它设为默认正式分支。它的价值在于提供一条完全不执行 solver 的长上下文 ICL 对照路径：Task1/3 用 28K anchored reasoning 直接输出候选，Task4 用多预算候选和 select-only ensemble 抑制字符级错误。若需要复现实验或展示长上下文 many-shot 行为，可设置 TASK134_BRANCH=longctx_manyshot；若追求当前正式包的稳定性，默认 solver_debug_loop

仍是主路径。

7.4 Task2/5/6 held-out 消融

表 8: Task2/5/6 held-out 消融完整排序

Task	variant	accuracy	correct/total
Task2	noun_rules	0.5667	68/120
Task2	bucket_balanced	0.5583	67/120
Task2	router	0.5667	68/120
Task2	context_rerank	0.5500	66/120
Task2	noun_biased	0.5417	65/120
Task5	ab_conservative_k8	0.7250	87/120
Task5	router_guarded	0.7000	84/120
Task5	ab_recall_k8	0.6917	83/120
Task5	baseline_t0	0.6333	76/120
Task5	strict_t0	0.6083	73/120
Task6	anchor	0.8667	104/120
Task6	router	0.8667	104/120
Task6	genrefirst	0.8417	101/120
Task6	hypothesis	0.8250	99/120
Task6	conservative	0.7917	95/120

表 9: Task2 count nouns/verbs 细粒度消融

variant	特征源	Acc@120	MAE	off-by-one
noun_rules	显式词性计数规则	0.5667	0.600	33
router	多候选 calibrated router	0.5667	0.592	34
bucket_balanced	答案数值分桶与分布校准	0.5583	0.617	32
context_rerank	词面/上下文相似示例	0.5500	0.575	39
noun_biased	名词偏置与任务元信息	0.5500	0.600	37

表 10: Task5 tweet sadness 混淆矩阵消融

variant	Acc	TP	TN	FP	FN	FPR	FNR
ab_conservative_k8	0.725	64	23	27	6	0.540	0.086
router_guarded	0.700	64	20	30	6	0.600	0.086
ab_recall_k8	0.692	66	17	33	4	0.660	0.057
baseline_t0	0.633	69	7	43	1	0.860	0.014
strict_t0	0.600	70	2	48	0	0.960	0.000

表 11: Task6 MNLI same genre 分标签消融

variant	保留信号	Acc@120	Y_acc	N_acc
anchor	Anchor 主体/场景锚点	0.8667	27/33	77/87
router	Anchor + 多候选校验	0.8667	27/33	77/87
genrefirst	优先依赖 genre	0.8417	26/33	75/87
hypothesis	Hypothesis 关系	0.8250	25/33	74/87
conservative	保守语义相似	0.7917	28/33	67/87

Task2 的结果差距很小，最高的 noun_rules 和四路 router 均为 68/120，bucket_balanced 为 67/120，context_rerank 为 66/120，说明这个任务的瓶颈不是示例数量不足，而是词性边界本身不稳定。答案值分桶显示 gold=1 的样本较易，noun_rules 为 31/34；gold=2/3 明显困难，分别为 5/22 和 3/19，说明主要误差来自中间计数边界。router 在 25/120 样本触发，最终仍以 current 为主。这个结论支持精简提交包当前采用的保守 legacy_router：保留多候选审计价值，但不让 router 无条件覆盖强主干。

从误差形态看，Task2 各分支 accuracy 集中在 0.550–0.567，差异只有 2 条左右；历史消融中 MAE 大致在 0.58–0.62，off-by-one 数量约 32–39 条。这说明 Task2 的主要问题是动名词、复合名词、修饰语和 caption 计数约定的边界，而不是上下文样例数量不足。继续扩大 router 覆盖面容易把边界歧义放大成系统性覆盖错误，因此正式精简包把 Task2 保持为四路候选的受限选择。

Task5 的消融差距更有解释价值。ab_conservative_k8 达到 87/120，比 baseline_t0 高 11 题，比 strict_t0 高 14 题，说明 A/B 标签压缩和保守边界提示显著优于普通二分类 prompt。混淆矩阵进一步说明为什么保守分支是主干：ab_conservative_k8 的 TP/TN/FP/FN 为 64/23/27/6，FPR 为 54.00%，FNR 为 8.57%；ab_recall_k8 虽把 FNR 降到 5.71%，但 FPR 升到 66.00%。因此 Task5 不能简单提高 Sad 召回，而要优先降低第三方事件、歌词、引用和讽刺造成的 Not sad 误报。guarded router 在 26/120 个样本上出现候选分歧，实际选择来源 current=117、aggressive=3，最终为 84/120。它没有超过保守主干，说明正式方案不能让 router 自由追逐 Sad 召回，而应把它定位为“只救明显第一人称痛苦”的低触发补丁。

Task6 的结果最稳定。anchor 主干和 router 都为 104/120，明显高于 genre-first 分支的 101/120、hypothesis 分支的 99/120 和 conservative 分支的 95/120。分标签结果显示 anchor 在 N 类为 77/87、Y 类为 27/33，整体最佳；conservative 对 Y 类略高，为 28/33，但 N 类退化到 67/87。这个排序说明 Task6 的核心判断不是单独的 genre 分类，也不是普通 hypothesis 判断，而是“同一 anchor 下的

genre-compatible hypothesis”。过度保守会把短 hypothesis 或简化句误判为 N；只做 genre-first 又会漏掉句子 2 与句子 1 的 same-source 关系。router 在 16/120 个样本上出现候选分歧，但选择来源仍为 current=120，说明 anchor 主干已经足够强，路由器没有破坏它；正式方案因此保留 anchor-first 作为主预测，同时保留其他分支用于审计和疑难样本对照。

综合来看，Task2/5/6 的消融给出的共同结论是：多候选不是为了做简单投票，而是为了暴露不同 prompt 的系统性偏差。只要 held-out 最强分支已经稳定，router 就应该低触发；只有候选分歧与任务规则高度一致时才替换。这个结论直接指导正式提交中的保守路由策略。

7.5 Task7 检索配置消融

表 12: Task7 semantic MMR 检索配置消融

rank	answer_rules	lambda	pool	k	chars	correct/total
1	preserve_articles	0.70	24	2	800	47/120
2	shortest	0.70	24	3	800	36/120
3	preserve_articles	0.70	24	3	800	36/120
4	shortest	0.75	36	3	1000	26/120

当前批次还生成了 7 个全量测试配置：k=1/2/3/4、不同 MMR lambda、不同 pool 和 char limit，均写出 500 行预测。config-B 的本地 sanity-check 仅用于粗验算，不等同官方 validation。更可靠的历史官方 validation grid 显示，Task7 的绝对准确率低于分类任务，这是预期现象：Jeopardy 属于开放短答案任务，答案空间巨大，且本方案不引入外部知识库，只能利用官方 examples 和模型内部知识。消融更重要的意义在于解释“怎样的检索上下文更少误导模型”。最优配置为 preserve-articles、lambda=0.70、pool=24、retrieval_k=2、char limit=800，命中 47/120。它比同样 lambda/pool 但 retrieval_k=3 的配置高 11 题，说明多放一个 reasoning case 并不一定提高效果；开放问答里，额外示例很可能提供相似但错误的实体、类别或答案格式，反而干扰最终答案。

answer rules 的差异也值得注意。在 retrieval_k=2 时 preserve-articles 最优，说明 Jeopardy 的答案有不少标题、短语或固定表达，机械删除 a/an/the 会损失信息；但当 retrieval_k=3 时 preserve-articles 与 shortest 都是 36/120，说明检索噪声已经超过答案规范本身的影响。第四组把 lambda 提到 0.75、pool 扩到 36、reasoning 截断放宽到 1000，反而降到 26/120，说明“更大候选池、更长推理、更高相关性权重”并非单调收益。对 Task7 来说，检索到的 reasoning case 必须既相关又克制，过长或过多会让模型复制旧答案、偏向相邻实体，或被无关 category 牵引。

因此 Task7 的正式策略不是追求最大检索量，而是把 reasoning bank 当作“格式和推理模式提示”：保留少量高质量案例，使用 MMR 去掉冗余，再用 answer rules 约束输出形态。这个消融也解释了为什么后续改进应优先优化语义标签和答案规范，而不是简单增加上下文长度。

7.6 Task8 pass@k 与 repair 消融

表 13: Task8 pass@k 与 repair 消融

variant	samples	smoke_ok	exec_ok	static_bad	function_match
full_passk_initial	166	142 (85.5%)	151	0	166
repair_round1	24	24 (100%)	24	0	24
final_validator	166	166 (100%)	166	0	166
sample20_k1	20	14 (70%)	15	2	18
sample20_k4	20	16 (80%)	18	0	20
sample20_k8	20	18 (90%)	18	0	20

Task8 的消融可以拆成“覆盖率”和“选择能力”两部分看。full_passk_initial 已经做到函数名匹配 166/166、静态违规 0，说明 prompt 中的函数名约束、Triton/CUDA 禁止项和 API hint 已经有效；但 smoke 只有 142/166，exec 为 151/166，说明仍有一部分候选虽然能编译、能定义函数，却在代表性输入上失败，或者 out、shape、dtype、特殊参数没有处理好。这里 smoke 比 exec 更严格，因为 exec 只说明代码能加载并找到函数，smoke 才会真实调用函数并检查基本行为。

repair_round1 针对 24 个未通过 smoke 的样本定向修复，24/24 smoke 和 exec 通过；最终 validator 达到 166/166。这个结果说明 repair 的收益不是来自“再随机生成一次”，而是来自带错误原因的反馈：例如缺少 out 处理、调用了不存在的函数、参数默认值不兼容、返回 tuple/张量形态不对等问题，都能在错误驱动 prompt 中被模型修正。更重要的是，ACCEPT_ONLY_SMOKE_OK_IMPROVEMENTS 让系统只接受从失败到通过的改进，避免 repair 把已有正确样本改坏。

小样本 k 值消融展示了 pass@k 的边际收益：历史 sample20 中，k=1 时 20 个样本中 smoke 14、exec 15、static bad 2、function match 18；k=4 时 smoke 提升到 16、exec 提升到 18，静态违规降为 0，函数名全匹配；k=8 时 smoke 进一步提升到 18，但 exec 仍为 18，说明继续增加候选主要提高可运行覆盖率，收益开始递减。当前 sample20 批次中 k=1/4/8 均达到 smoke=20/20、exec=20/20、static bad=0、function match=20/20，说明最终包本身已经稳定。正式全量采用更高 k 加 validator，而不是只靠 k，是因为多候选只提供“可能正确的答案池”，最终质量取决于验证器能否选出真正可执行、签名正确、行为稳定的候选。

从工程角度看，Task8 消融证明了三个环节缺一不可：pass@k 扩大候选覆盖，validator 把代码生成转化为可判定选择，repair 处理剩余失败样本。若只有 pass@k 没有 validator，模型会产生多个候选但无法可靠排序；若只有 validator 没有 repair，初始 24 个 smoke 失败会保留；若只有 repair 没有 accept-only-improvement 门控，修复轮可能引入回归。因此最终方案采用生成、验证、修复、再验证的闭环。

7.7 跨任务综合分析与提交策略

消融实验整体支持六条跨任务发现。

第一，程序化求解是规则确定性任务的决定性组件。Task1/3 的 solver 分支相对自由 ICL 提升

超过 82 个百分点，Task4 的 strict prompt 和 structured examples 也显著低于 solver。机理是 LLM 自由生成倾向于“理解并重写”，而规则题要求逐元素、逐字符执行。

第二，debug loop 的价值主要是工程保险。 Task1/3 中完整分支和去掉 debug loop 的历史结果都能达到 500/500，但 debug loop 在 solver 输出格式错误或边界样本失败时提供可解释回退，是防止采样退化的保险层。

第三，分类任务的性能边界由语言学边界建模决定。 Task2 的分支差距很小，Task6 的 anchor-first 明显优于 conservative，说明关键不是继续增加上下文长度，而是更准确地建模词性、genre anchor 和语义归因边界。

第四，router 的最优定位是低触发纠偏，而不是替代强主干。 Task2 中四路 router 与 noun rules 持平，Task5 的 guarded router 低于 conservative A/B 主干，Task6 router 全部保留 current。这说明 router 应当服务于候选审计和高置信纠偏，不能无条件覆盖强分支。

第五，开放短答案存在上下文噪声拐点。 Task7 中 retrieval_k=2 明显优于 k=3，说明更多 reasoning case 会引入相邻实体、错误类别和答案格式噪声。开放答案更需要控制检索质量，而不是追求最长上下文。

第六，代码质量来源于验证闭环而非候选数量。 Task8 从初始 142/166 smoke 通过到最终 166/166，关键提升来自 validator 定位失败和 repair 定向修复；pass@k 只是提供候选池，不能替代验证器。

表 14: 任务级提交策略建议

任务	建议保留策略	不建议做法	依据
Task1	Solver + debug loop 保险层	退回逐样本 ICL	规则确定性强，solver 提升决定性
Task2	noun rules/current 强主干 + calibrated router	扩大激进 router 覆盖	词性边界歧义主导误差
Task3	Solver + list 格式规范	让模型直接生成列表	Collatz 规则和列表格式强耦合
Task4	程序化字符拼接；可选 longctx many-shot 对照	依赖 strict prompt 或结构示例	字符级保真要求可执行化
Task5	conservative A/B 主干 + 低触发 router	把所有负面语气召回为 Sad	FPR 成本高，误报来自引用/讽刺/第三方事件
Task6	anchor-first 三层门控	仅按 genre 或表面语义判断	anchor 主干 held-out 最稳
Task7	k=2、lambda=0.70、pool=24 的克制检索	盲目增大 k/pool/char limit	检索噪声存在拐点
Task8	pass@k + validator + repair 闭环	接受未经 smoke/exec 验证代码	质量由验证闭环保障

8 可复现性、合规与工程控制

8.1 赛题要求对齐

表 15: 官方要求与本方案对应关系

官方要求或评分点	本方案对应设计	报告与代码证据
仅允许使用 Qwen3-4B	所有生成、路由、修复和候选选择均调用同一个 Qwen3-4B 服务	configs/llm_config.yaml、scripts/start_service.sh、第 2 节服务配置
必须使用 FlagScale	由 scripts / start _ service . sh 按 FLAGSCALE_DIR 调用 FlagScale 的 run.py 拉起 OpenAI-compatible 推理服务，算法代码只访问本地 API	第 2.1 节服务层搭建
仅允许使用官方数据集	输入只读取当前提交目录中的官方输入副本 input/，包含 task definition、examples、test samples	第 1.1 节总流程、第 8.3 节约束
全部 8 个任务完整推理	run _ full _ inference . sh 顺序运行 Task1–Task8，并输出 8 个根目录 JSONL	第 2.2 节推理编排、第 7.1 节质量信号
提示策略与输出约束 (技术方案 20%)	共享 prompt 骨架、任务契约、输出空间压缩；规则题输出 solver，分类题压缩为标签或候选字母，Task8 输出 PyTorch fallback	第 4 节
超长上下文构造（技术方案 40%）	Task1–Task7 主生成阶段补足约 30K tokens，Task8 补足约 16K tokens；示例按结构相似、标签平衡、MMR、共享前缀和动态尾部组织	第 3 节、第 6 节
自动多轮交互与一致性 (技术方案 40%)	solver debug loop、候选 router、Task8 pass@k + validator + repair 形成自动闭环	第 5 节、第 6.9 节
可复现源代码与说明	提供 README.md、requirements.txt、run _ full _ inference . sh、scripts / start _ service.sh、src/prepare_submission.py 和 verify_submission.py	第 8.3 节

8.2 评审指标对齐

官方预测结果评分中，Task1–Task7 主要考察逐样本标注 accuracy，因此本方案分别围绕规则可执行化、词性边界控制、标签先验校准和开放短答案规范化来减少逐样本错误。Task8 使用逻辑一致性指标，重点同时考察生成代码能否被成功调用以及能否正确执行计算；本方案的函数名匹

配、静态检查、编译加载、CPU smoke test 和执行验证分别对应 call 与 execution 两类信号，最终只接受通过 validator 的候选进入提交包。

8.3 合规与复现材料

本项目遵守以下约束：

1. 仅使用 Qwen3-4B 进行候选生成、候选路由、错误修复和辅助判断。
2. 使用 FlagScale 作为模型加载与运行框架。
3. 不进行微调，不加载 LoRA，不保存训练 checkpoint。
4. 不调用外部模型、外部 API、外部检索服务或外部数据。
5. 不使用 Qwen3-4B 以外的模型为比赛任务生成、筛选或修正标注结果。
6. 不读取历史提交 zip、历史 final outputs 或人工答案作为预测输入。
7. 消融实验只用于解释方法，不参与正式测试集答案生成。
8. src/prepare_submission.py 只做 JSONL 规范化和 zip 打包，不做面向标签的答案修正。
9. 复现材料包含环境文件、模型服务脚本、完整推理脚本、提交打包脚本和提交校验脚本。
10. 重大实验和方法更改通过 README.md、manifest.json、docs/ 补充材料及脚本内参数记录追溯。

9 团队介绍

9.1 团队概述

本队 TrailBlazers 由三名成员组成，覆盖自然语言处理与长上下文学习、智能体与代码生成、数据驱动实验与工程交付等能力，与赛道三“超长上下文场景下大语言模型自动数据标注”的任务要求相匹配。团队以 Qwen3-4B 与 FlagScale 为统一底座，构建“示例组织-提示策略-推理校验-提交复现”的完整技术闭环，在八个评测任务上分别采用求解器合成、校准路由、语义 MMR 示例选择与 Task8 多轮 pass@k 修复等策略。团队具备从方案设计、算力部署到榜单提交与开源交付的协作经验，由队长黄莉莉同学统筹实验分析、代码生成与可复现流程，邢翔同学承担实验设计与核心代码实现，张文婧同学负责系统部署与技术报告。

9.2 成员简介

黄莉莉，队长，实验分析与代码生成。广东工业大学计算机科学与技术硕士，研究领域为域适应、迁移学习、计算机视觉、自然语言处理与 RAG。具备智能体运行时、RAG 全链路与大规模数据验证经验；熟悉 Spring AI、LangGraph 多智能体协作、Flask、PySpark 与 pytest 数据质量框架，擅长 hold-out 评测设计、消融对比、实验记录与可复现流程管理。

邢翔，成员，实验设计与代码实现。扬州大学计算机科学与技术本科，研究方向为自然语言处理、提示学习、检索增强生成（RAG）与机器学习应用。具备从数据处理、模型调用、检索增强、实验评测到前后端系统部署的 AI 应用全链路能力；独立完成政府文件 RAG 智能问答、Agent RAG 自主检索代理等系统，熟悉文档分块、向量检索、重排序、多模型 API 接入与流式输出，具

备长文本检索与提示调优的实战经验。邢翔同学还围绕中文电竞弹幕恶意用户早期预测撰写论文 *An Early Malicious Users Prediction Benchmark for Chinese Esports via Bullet Chats*，工作涉及大规模弹幕数据清洗、LLM 多 prompt 辅助标注、人工复核、提示学习与用户级风险预测基准构建，与本赛季的数据标注、提示策略和可复现实验设计高度相关。

张文婧，系统部署与技术报告。香港中文大学（深圳）金融数学硕士在读，苏州大学统计学本科。聚焦大模型业务产品化、数据建模与智能体辅助研发，熟悉 Python、SQL、R 及 Cursor 等 Agent 工具；适合承担代码类标注任务与推理服务稳定性保障工作。

9.3 团队分工

团队按“队长统筹 + 任务模块负责制”推进。黄莉莉同学负责整体协同、实验分析与代码生成闭环；邢翔同学承担实验设计与核心代码实现的主要工作；张文婧同学负责系统部署、技术报告与交付材料整理。关键版本由全员联合跑通并完成校验后提交。

表 16: 团队分工

模块	主要内容	主要负责人	协作成员
总体方案与主线推理	长上下文 ICL 流水线设计、统一推理入口、八任务调度与方案定型	邢翔	黄莉莉
Task1/3/4	求解器合成与调试闭环、输出格式校验	黄莉莉	邢翔
Task2/5/6	提示工程、A/B 标注器、校准路由与守卫路由	邢翔	张文婧
Task7	推理库构建、语义 MMR 示例选择、候选路由与网格实验	邢翔	张文婧
Task8	pass@k、校验器、修复流程与 PyTorch 回退实现	黄莉莉	邢翔
推理服务与算力	FlagScale 启停、30K/16K 上下文配置、前缀缓存与并发参数	黄莉莉	邢翔
实验与评测	hold-out 划分、消融对比、上下文审计、结果汇总	张文婧	邢翔
交付与文档	JSONL 打包、提交校验、技术报告与系统演示材料	张文婧	全员

团队采用周计划与任务看板推进；黄莉莉同学作为队长负责实验分析、代码生成类任务与可复现流程管理；邢翔同学负责提示-路由类实验设计、核心代码实现与长文本检索调优；张文婧同学负责推理服务部署、技术报告和提交质检。正式提交版本须由三人联合执行全任务推理脚本并通过校验后，方可上传平台。

9.4 团队优势与稳定性说明

能力互补。黄莉莉同学作为队长，覆盖实验分析、代码生成、智能体运行时、RAG 全链路与数据质量验证；邢翔同学作为成员，覆盖 NLP/RAG、提示学习、长上下文检索增强、实验设计与核心代码实现；张文婧同学补强系统部署、数据建模、技术报告和推理服务稳定性保障，共同对应赛题在提示工程、上下文构造、自动标注与可复现实验等方面的技术要求。

分工清晰、可互相备份。团队按任务类型纵向切分、按“算法-系统-实验”横向协同，职责边界明确；核心模块均有协作备份，降低单点风险。

成员稳定。三人自 2026 年 1 月组队参赛以来，持续参与文献调研、基线修复、官方提交迭代与技术报告撰写，团队组成未发生变动。

10 Demo 展示

为便于评审和答辩展示，项目新增独立静态展示页：

docs/网页demo/system_showcase.html

页面不依赖后端服务，直接在浏览器打开即可。Demo 以仪表盘形式展示系统运行阶段、三类算法策略、Task1–Task8 进度条、Task8 validator 指标和关键消融结论；其中“演示推理进度”按钮会模拟从服务检查、solver、候选路由、Task7 检索、Task8 pass@k 到提交打包的完整进度，用于说明真实推理系统的状态流转。

图 1–5 展示了 Demo 页面的主要交互状态。页面上方用于呈现方案概览和关键指标，中部展示任务策略、进度状态和验证结果，下方补充消融结论与运行审计信息。该 Demo 的目标不是替代正式推理脚本，而是在答辩场景中把长上下文构造、候选路由、Task8 验证和打包审计等工程流程可视化。

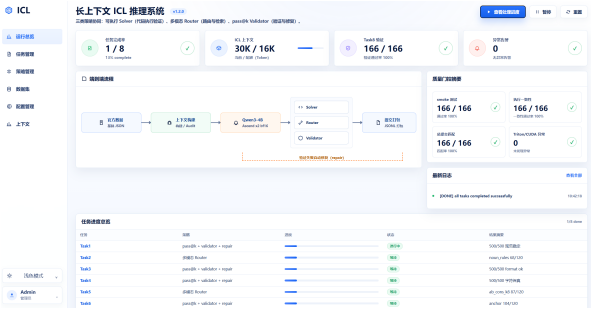


图 1: Demo 页面概览与核心指标

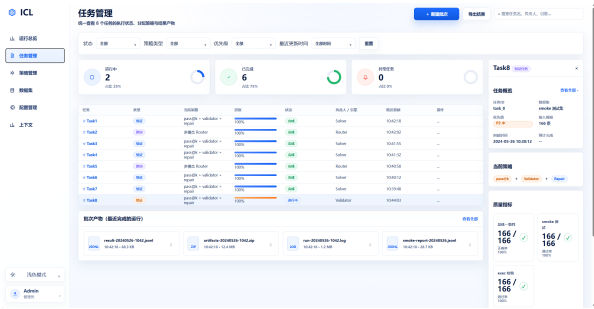


图 2: Demo 推理阶段与任务进度展示

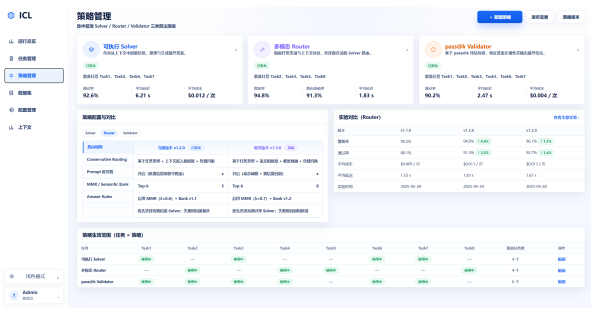


图 3: Demo 多策略流程与候选路由展示

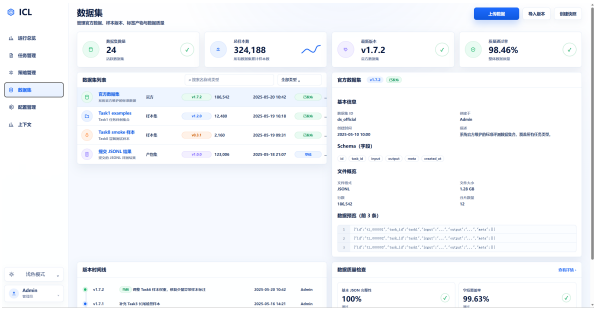


图 4: Demo Task8 验证指标与执行门控展示

11 系统架构与关键流程图

图 7 展示了从官方数据读取、长上下文构造、Qwen3-4B/FlagScale 服务，到三类任务引擎、验证审计和最终打包的总体链路。

图 8 展示了 24K 共享 ICL 前缀、动态相关示例尾部、当前 query 和输出契约之间的位置关系。该结构用于同时兼顾 prefix cache 复用和样本相关信息靠近输出位置。

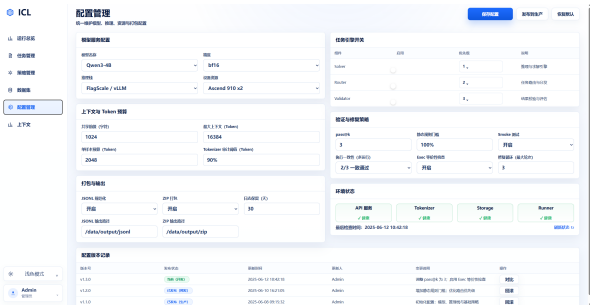


图 5: Demo 消融结论与审计信息展示

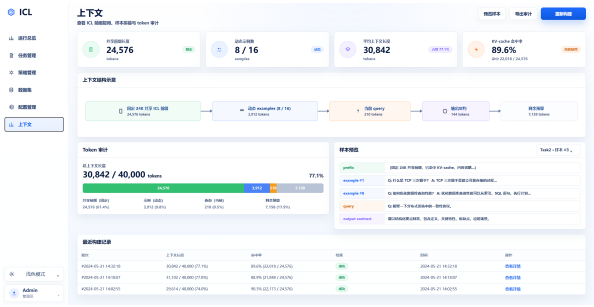


图 6: Demo 完整运行状态展示



图 7: 长上下文 LLM 自动标注系统技术架构



图 8: 长上下文 Prompt 布局与 KV cache 复用

图 9 将八项任务映射到三类策略：Task1/3/4 使用可执行 solver，Task2/5/6/7 使用多候选路由，Task8 使用 pass@k 验证与修复闭环。

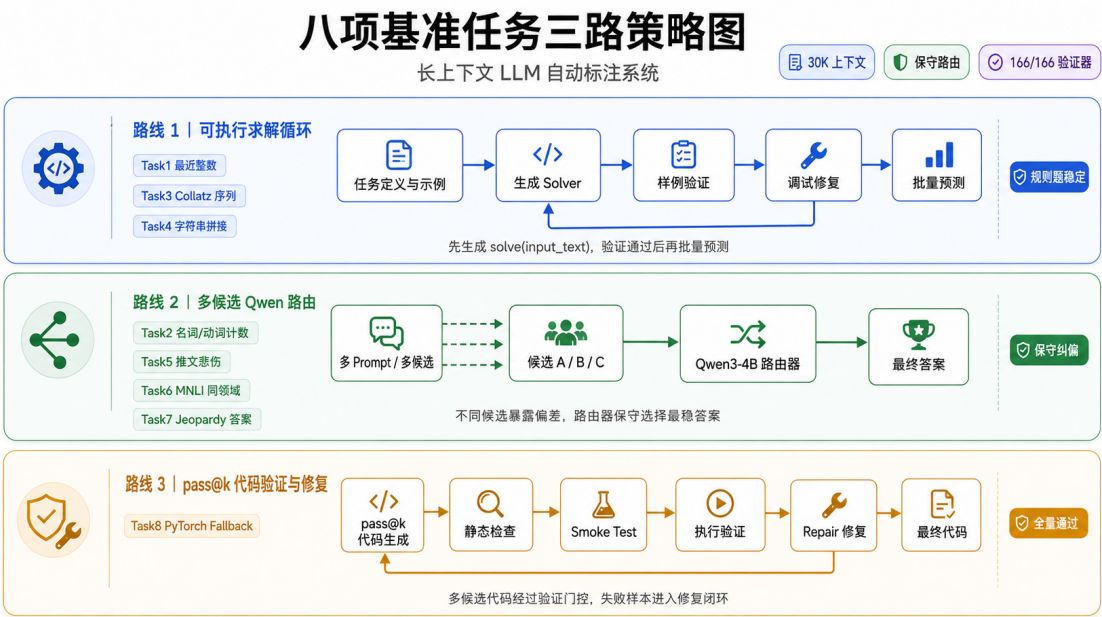


图 9: 八项基准任务的三路策略映射

图 10 展示了 Task8 的候选生成、静态门控、函数名匹配、编译、smoke、执行验证、repair 和 final gate 的完整闭环。

参考文献

[1] Tom B. Brown et al. Language Models are Few-Shot Learners. NeurIPS, 2020.

[2] Rishabh Agarwal et al. Many-Shot In-Context Learning. 2024.

[3] Nelson F. Liu et al. Lost in the Middle: How Language Models Use Long Contexts. TACL, 2024.

[4] Jiachang Liu et al. What Makes Good In-Context Examples for GPT-3? DeeLIO, 2022.

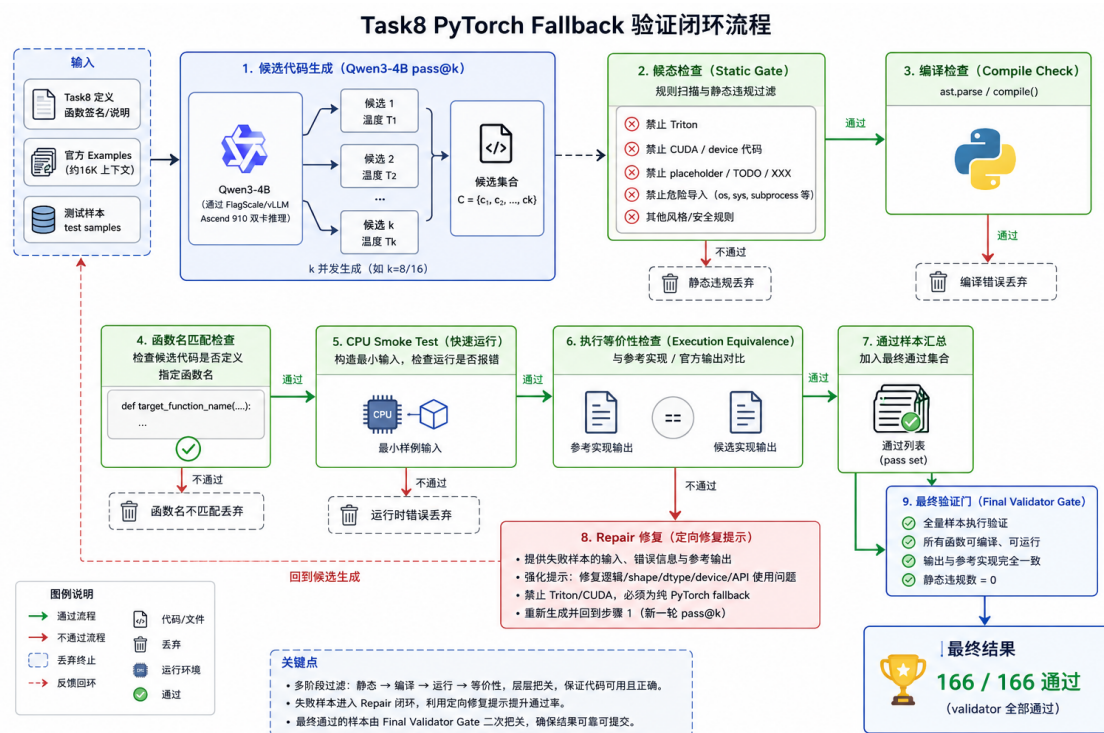
[5] Tony Z. Zhao et al. Calibrate Before Use: Improving Few-Shot Performance of Language Models. ICML, 2021.

[6] Xuezhi Wang et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR, 2023.

[7] Omar Khattab et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. ICLR, 2024.

12 局限与后续改进

Task2 仍受英语词性边界影响，动名词、复合名词、专名和修饰语判断容易让不同 prompt 产生系统性偏差。当前精简包采用四路候选 router，仍只能在候选整数之间做保守选择；后续可以在



不引入外部数据的前提下，让模型先生成 **token-level** 结构化分析，再把结构化证据交给 **router** 选择整数候选。针对 **off-by-one** 错误，还可以单独设计差值校正和高风险结构审计，例如 **ing**、复合名词、颜色修饰词和介词短语。

Task5 对讽刺、歌词、引用和隐晦情绪仍敏感，**held-out** 中 **FPR** 仍然偏高。后续应继续强化“作者本人状态”和“第三方负面事件”的对比示例，而不是单纯提高 **Sad** 召回；也可以把引用/歌词、第三方事件、讽刺、**anger-vs-sad** 等误报来源拆成细分审计标签，用于指导 **A/B prompt** 和 **router** 的低触发修正。

Task7 的开放答案上限受模型内部知识和 **official examples** 覆盖影响较大。后续应先确认 **oracle recall**，即正确答案是否已经出现在候选或近邻 **reasoning case** 中，再优化生成策略；可以继续优化 **reasoning bank** 的压缩粒度、语义标签字段、答案规范规则和 **MMR** 多样性目标，避免简单增大检索数量。

Task8 本地 **validator** 已全通过，但隐藏评测可能覆盖更极端的 **shape**、**dtype**、**device** 或数值稳定性。后续应扩展本地 **smoke/exec** 输入族，覆盖更多边界分支；同时继续保持 **ACCEPT_ONLY_SMOKE_OK_IMPROVEMENTS** 一类防回归门控，并针对 **shape mismatch**、**dtype** 不一致、**out** 参数、返回 **tuple**/张量形态等常见失败模式设计更精细的 **repair prompt**。

更具体地，Task2 后续应优先构建 **token-level** 词性结构化证据，把每个候选名词/动词的边界、修饰关系和排除理由显式写出，再交给规则融合或 **router** 选择整数候选；同时针对当前 32–39 条 **off-by-one** 错误建立差值校正机制，而不是继续盲目扩展上下文。Task5 后续应单独设计讽刺、歌词引用、第三方负面事件和 **anger-vs-sad** 的对比审计集，降低 **False Positive**，而不是单纯增加投票轮数。Task7 后续需要先估计 **oracle recall**：如果正确答案没有出现在候选或近邻 **reasoning case** 中，应扩展 **reasoning bank** 覆盖；如果已经出现，则优先优化答案规范、冠词保留和候选路由。Task8

后续应扩展 **smoke** 输入族，覆盖高维 **shape**、非默认 **dtype**、**device** 迁移、**out** 参数和数值稳定性边界，并保留只接受验证更优候选的防回归门控。

跨任务看，后续工作有四条方向。第一，统一验证闭环框架，把 **Task1–Task7** 的 **solver/router** 审计与 **Task8** 的 **pass@k + repair** 思路抽象成统一的候选生成、验证、修复、再验证接口。第二，在仍只使用 **Qwen3-4B** 和官方数据的当前约束下，探索自适应上下文窗口：根据样本难度、候选分歧和 **router** 置信度动态调整示例数量，而不是固定 **30K/16K**。第三，继续完善同一模型内的多提示、多候选协同，这与本次赛题约束一致，也更容易复现和审计。第四，作为更远期设想，如果后续规则或应用场景允许多模型参与，可以探索 **Qwen3-4B** 与轻量级专用模型的协同，例如用小模型辅助词性结构分析、情绪边界初筛、代码候选静态风险评分或 **router** 置信度估计；这类方向只作为未来扩展，不属于当前提交使用的模型范围。

总体而言，本方案在官方约束下建立了清晰的任务拆解、长上下文组织、多候选选择、可执行验证和推理性能优化体系。它既能解释最终结果的主要来源，也能支撑后续复现、答辩展示和进一步迭代。